

Software Development and Coding

Topics that will cover:

- Programming Principles (DRY, KISS, SOLID)
- Code Reviews
- Version Control Systems (Git basics)
- Coding Standards & Guidelines

1. Programming Principles

- These are **best practices** for writing **good, clean code**.
- The **most common ones**:

DRY, KISS, SOLID.

1. DRY — Don't Repeat Yourself

What it means:

Don't write the **same code again and again**.

Why:

If you repeat code and later need to fix something, you'll have to fix it in **many places**. That's risky and time-wasting.

Example:

Bad:

You write the same *validateEmail()* function in 5 files.

Good:

Write *validateEmail()* once, then reuse it everywhere.

2. KISS — Keep It Simple, Stupid

What it means:

Write your code as **simple and clear** as possible.

Don't make it fancy for no reason.

Why:

Simple code is **easier to read**, test, and debug.

Example:

Bad:

Using a super complicated trick to sort numbers in 5 lines nobody understands.

Good:

Use a clear loop or built-in sort method.

3. SOLID

What it means:

SOLID is a set of 5 principles that help you design better **OOP (Object-Oriented Programming)** code.

S — Single Responsibility Principle (SRP)

One class = one job only.

Example: StepCounterService only counts steps. It shouldn't handle database too.

O — Open/Closed Principle (OCP)

Software should be **open for extension but closed for modification**.

Example: Add new workout types **without changing** old code. Use interfaces.

L — Liskov Substitution Principle (LSP)

Subclasses should work the same way as their parent.

Example: If you have a Shape class, a Circle or Square subclass should work wherever Shape works.

I — Interface Segregation Principle (ISP)

Don't force a class to depend on methods it doesn't use.

Example: If you have an interface Printable, don't put unrelated stuff like SaveToFile in it.

D — Dependency Inversion Principle (DIP)

Depend on **abstractions**, not concrete stuff.

Example: Code should use an interface, not a hard-coded class. Makes it flexible to swap parts.

2.Code Reviews

What is it?

Before finalizing code, another developer **checks your code** for:

- Bugs
- Style problems
- Logic errors
- Better ways to do it

Why do it?

- Improves quality
- Catches mistakes early
- Teaches team members better coding habits

How it works:

1. Developer writes code → pushes it.
2. Another developer checks (comments, suggests changes).
3. Developer fixes issues → final version is approved.

Real-life:

It's like proofreading an essay before submitting it!

3. Version Control Systems (VCS)

What is it?

A VCS **keeps track of every change** in your code.

Why:

- You can go back to an old version if something breaks.
- Work in teams without overwriting each other's code.
- Manage multiple versions or features at once.

Popular Tool: Git

Basic Git Commands:

- **git init** — start a repo.
- **git add** — add files.
- **git commit** — save changes.
- **git push** — upload changes to GitHub.
- **git pull** — get latest changes.
- **git branch** — make a new line of development.
- **git merge** — combine branches.

Real-life:

It's like Google Docs version history but for code!

4. Coding Standards & Guidelines

What is it?

Rules and styles for writing neat, readable, and consistent code.

Why:

- Makes it easy for everyone to read and understand.
- Prevents silly mistakes.
- Keeps the whole project clean.

Examples:

- Use clear names: `stepCounter` not `sc`.
- Use consistent indentation: 2 or 4 spaces.
- Add comments where needed.
- Follow naming conventions: e.g., `CamelCase` for classes, `snake_case` for variables.
- Avoid long functions — break them into smaller ones.

Real-life:

It's like following grammar rules when you write an essay . it looks good and makes sense to others!